Módulo 3

Compilación a JavaScript

Curso: TypeScript

45 min

Objetivos de Aprendizaje

- Comprender el **proceso de compilación** de TypeScript a JavaScript
- Dominar la configuración de **tsconfig.json**
- Utilizar **scripts de compilación** y vigilancia
- Ejecutar código TypeScript **compilado**

1. El Compilador TypeScript

¿Qué es tsc?

¿Qué es tsc?

tsc = TypeScript Compiler

- Transforma código TypeScript a JavaScript estándar
- Elimina los tipos en tiempo de compilación
- Genera archivos `.js` y opcionalmente `.js.map`

Proceso de Compilación

```
archivo.ts ──► tsc ──► archivo.js ──► Node.js / Navegador
```

Flujo: TypeScript → Compilador → JavaScript → Ejecución

¿Por Qué Compilar?

Razón	Explicación
Compatibilidad	Los navegadores no entienden TypeScript, solo JavaScript
Tamaño	El código compilado es más pequeño (sin tipos)
Portabilidad	Compatible hacia atrás con cualquier entorno JS

2. tsconfig.json

Guía Completa de Configuración

¿Qué es tsconfig.json?

- Archivo de configuración del compilador TypeScript
- Define cómo se compila el código
- Personaliza el comportamiento de `tsc`

Ubicación: Raíz del proyecto TypeScript

Estructura Básica

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

Opción target: Versión de JavaScript

Valor	Descripción	Compatibilidad
`ES3`	JavaScript más antiguo	IE6+
`ES5`	JavaScript moderno	Todos los navegadores
`ES2015` / `ES6`	ES6	Navegadores modernos
`ES2020`	ES2020	Navegadores recientes
`ESNext`	Última versión	Entornos modernos

Opciones Principales

Opción	Descripción
`target`	Versión de JavaScript destino
`module`	Sistema de módulos (commonjs, esnext)
`outDir`	Carpeta de salida
`rootDir`	Carpeta raíz del código fuente
`strict`	Habilitar modo estricto
`lib`	Bibliotecas incluidas
`esModuleInterop`	Interoperabilidad entre módulos
`skipLibCheck`	Omitir verificación de tipos en librerías
`declaration`	Generar archivos .d.ts
`sourceMap`	Generar source maps

Archivos a Incluir/Excluir

```
{
  "compilerOptions": { ... },

  // Archivos a incluir en la compilación
  "include": [
    "src/**/*",      // Todo en src/
    "*.ts"           // Archivos .ts en raíz
  ],

  // Archivos a excluir de la compilación
  "exclude": [
    "node_modules",  // Dependencias
    "dist",         // Salida anterior
    "**/*.test.ts", // Tests
    "*.spec.ts"
  ]
}
```

Conclusión: tsconfig.json

- **tsconfig.json** es el corazón de la compilación
- Configura `target` según tu entorno destino
- Usa `strict: true` para mejor calidad de código
- `include` y `exclude` controlan qué archivos se compilan

Referencias:

- Documentación tsconfig.json
- Opciones del compilador

Lab

Configuración y Compilación

Ejercicio Práctico

Objetivo: Configurar un proyecto TypeScript y compilarlo a JavaScript

Continuaremos con el Caso 1: Sistema de Gestión de Tareas

Pasos para el Lab

1. Crear archivo `tsconfig.json` con configuración básica
2. Configurar `target: ES2020` y `module: commonjs`
3. Establecer `outDir: "./dist"` y `rootDir: "./src"`
4. Habilitar `strict: true` y `sourceMap: true`
5. Ejecutar `npx tsc` para compilar
6. Verificar los archivos generados en `dist/`

Configuración Esperada

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "declaration": true,
    "sourceMap": true,
    "lib": ["ES2020"]
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

Verificación del Lab

- Ejecutar `npx tsc` sin errores
- Archivos `.js` generados en `dist/`
- Archivos `.js.map` generados
- Archivos `.d.ts` generados (si aplica)

¡Excelente! Tu proyecto ahora compila correctamente

4. Scripts de Compilación

Automatiza tu flujo de trabajo

Scripts en package.json

```
{
  "scripts": {
    "build": "tsc",
    "dev": "tsc --watch",
    "start": "node dist/index.js",
    "clean": "rm -rf dist"
  }
}
```

Ejecutar Compilación

- **Compilar una vez:** `npm run build`
- **Modo vigilancia:** `npm run dev`
- **Ejecutar:** `npm start`

5. Empaquetadores

Herramientas de Build para proyectos modernos

¿Por qué necesitamos empaquetadores?

- Múltiples archivos TypeScript que necesitan unirse
- Optimización: minificación, tree-shaking
- Hot Module Replacement (HMR)
- Carga de assets: imágenes, CSS, fuentes

Empaquetadores Populares

Empaquetador	Características	Cuándo usarlo
Vite	Rápido, HMR instantáneo	Proyectos modernos
Webpack	Flexible, ecosistema huge	Proyectos complejos
esbuild	Extremadamente rápido	Build scripts

Vite: El recomendado

```
npm create vite@latest mi-proyecto -- --template react-ts
```

- ⚡ Compilación en milisegundos
- 🔥 HMR instantáneo
- 📦 Optimización automática

Lab 2: Compilar Caso 1

Task Manager

Compilar el Proyecto

```
# En la carpeta task-manager
npm run build
node dist/index.js
```

Objetivo: Compilar el proyecto del Caso 1 y ejecutarlo

Tips y Gotchas

Errores comunes y mejores prácticas

✅ Tips

- Usa `strict: true` desde el inicio
- `target` debe coincidir con tus requisitos
- `outDir` diferente de `rootDir`
- `declaration` genera .d.ts para paquetes

⚠️ Gotchas

- Cambios en tsconfig no se recargan - reinicia VS Code
- `exclude` no garantiza exclusión total
- `moduleResolution: "node"` vs `"bundler"`
- `tsc -w` no detecta nuevos archivos

Ejercicio Integrador

Caso 2: Biblioteca

Objetivo

Configurar el proyecto completo integrando M1, M2 y M3

Trabajo en equipo: 2-4 personas

Pasos

1. Crear proyecto: `mkdir biblioteca`
2. Inicializar: `npm init -y && npm install -D typescript`
3. Configurar Git: `git init`
4. Crear `tsconfig.json` completo
5. Compilar y ejecutar

Configuración Esperada

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "declaration": true,
    "sourceMap": true
  },
  "include": ["src/**/*"]
}
```

Resumen

Puntos Clave

- **tsc** compila TypeScript a JavaScript
- **tsconfig.json** configura el comportamiento del compilador
- `target` define la versión de JavaScript destino
- `strict: true` habilita verificaciones estrictas
- `outDir` y `rootDir` controlan la estructura de salida
- Compilar con `npx tsc` o scripts npm

Referencias

Recursos Adicionales

- Documentación oficial de TypeScript
- Guía de tsconfig.json
- TypeScript Playground
- Opciones del compilador

Gracias!

Módulo 3 Completado

Siguiente: Módulo 4 - Tipos de datos básicos